Innomatics Research Labs

Data Science Internship — February 2026

# AI Resume Screening System

## with LangChain & LangSmith Tracing

---

Assignment Report & Technical Documentation

Submitted by: Prasad Bairagi

Date: April 15, 2026

Task: GenAI — AI Resume Screening System with Tracing

# 1. Project Overview

This report documents the design, implementation, and results of an AI-powered Resume Screening System built as Task 6 of the Innomatics Research Labs Data Science Internship (February 2026 cohort). The system leverages LangChain LCEL pipelines, Groq's LLaMA-3.3-70B language model, and LangSmith for end-to-end tracing and debugging.

## 1.1 Objective

- Build an AI pipeline that evaluates resumes against a job description
- Use LangChain (LCEL) and LangSmith for modular, traceable pipeline execution
- Produce explainable outputs: Fit Score + Grade + Recommendation + Reasoning
- Demonstrate LangSmith tracing with at least 3 runs (Strong, Average, Weak)
- Implement anti-hallucination prompting — no assumed skills

## 1.2 Tools & Technologies

| Component | Technology Used | Purpose |
|---|---|---|
| Language | Python 3.10+ | Core implementation |
| Pipeline Framework | LangChain (LCEL) | Chain orchestration |
| LLM Provider | Groq — LLaMA-3.3-70B | Fast, free inference |
| Tracing & Monitoring | LangSmith | Debugging & observability |
| Notebook Environment | Jupyter Notebook | Development & submission |
| Prompt Design | PromptTemplate + FewShotPromptTemplate | Structured prompting |

# 2. System Architecture

The system follows a modular pipeline architecture as required by the assignment. All modules are implemented inline within the Jupyter Notebook to support Colab/Jupyter execution, while maintaining the logical structure of a production codebase.

## 2.1 Modular Structure

The project is organized to mirror the required prompts/ → chains/ → main.py layout:

| Module | File (Logical) | Responsibility |
|---|---|---|
| LLM Config | utils/config.py | get_llm() factory — returns ChatGroq instance |
| Extraction Prompt | prompts/extraction_prompt.py | PromptTemplate for skill/experience extraction |
| Matching Prompt | prompts/matching_prompt.py | PromptTemplate for JD vs. profile comparison |

| Module | File (Logical) | Responsibility |
|---|---|---|
| Scoring Prompt | prompts/scoring_prompt.py | PromptTemplate for 0–100 scoring with rubric |
| Extraction Chain | chains/extraction_chain.py | extraction_prompt \| llm (temp=0.0) |
| Matching Chain | chains/matching_chain.py | matching_prompt \| llm (temp=0.0) |
| Scoring Chain | chains/scoring_chain.py | scoring_prompt \| llm (temp=0.1) |
| Main Pipeline | main.py | screen_resume() orchestrator function |

### 2.2 Pipeline Flow

The full pipeline follows the pattern required in the assignment:

```
Resume Text
    │
    ▼
Step 1: Skill Extraction  →  extraction_prompt | LLM(temp=0.0)
    │  Output: JSON { name, years_experience, skills, tools, key_projects }
    ▼
Step 2: Matching Logic    →  matching_prompt   | LLM(temp=0.0)
    │  Output: JSON { matched_skills, missing_skills, overall_match }
    ▼
Step 3: Scoring + Explain →  scoring_prompt    | LLM(temp=0.1)
    │  Output: JSON { skills_score, total_score, grade, recommendation, explanation }
    ▼
LangSmith Trace (automatic via LANGCHAIN_TRACING_V2=true)
```

## 3. Input Data

### 3.1 Job Description

A Data Scientist role at TechCorp Analytics was used as the target position. Key requirements included:

- Python (5+ years), SQL, pandas, numpy
- Machine Learning: scikit-learn, XGBoost, LightGBM
- Deep Learning: TensorFlow or PyTorch
- Cloud platforms: AWS, GCP, or Azure
- MLOps: MLflow, Docker, Kubernetes
- Statistics: hypothesis testing, A/B testing
- Experience: 3–5 years in a data science role
- Education: Bachelor's or Master's in CS, Statistics, or related field

### 3.2 Candidate Profiles

| Candidate | Type | Experience | Education | Key Strength / Weakness |
|---|---|---|---|---|
| Priya Sharma | Strong | 4 years, Flipkart AI Labs | M.Tech CS, IIT Bombay | Full JD match — ML, DL, AWS, MLOps, Statistics |
| Rahul Verma | Average | 2.5 years, Infosys | B.Tech IT, VIT University | Partial match — Python & scikit-learn, no DL/MLOps |
| Anil Kumar | Weak | 6 months web dev intern | B.Com, Delhi University | No ML/DS skills — web dev background only |

## 4. Prompt Engineering

### 4.1 Extraction Prompt

The extraction prompt instructs the LLM to extract candidate information exclusively from the provided resume text, with explicit anti-hallucination instructions:

```
You are a professional resume parser. Extract information ONLY from the
resume text provided. Do NOT assume, infer, or add any skills, tools, or
experience NOT explicitly mentioned.

Return EXACTLY this JSON: { name, years_experience, education, skills,
                            tools, key_projects }
```

Temperature set to 0.0 for deterministic, consistent extraction with no creativity.

### 4.2 Matching Prompt

The matching prompt performs a structured gap analysis between the extracted profile and the job description:

```
You are a senior recruiter performing a skill-gap analysis.
Compare the candidate profile ONLY against the job requirements.
Do NOT award credit for skills not listed in the candidate profile.

Return: { matched_skills, missing_skills, experience_match,
          education_match, overall_match }
```

### 4.3 Scoring Prompt

The scoring prompt uses an explicit rubric to assign a 0–100 score, ensuring consistent and explainable outputs:

| Scoring Dimension | Max Points | Description |
|---|---|---|
| Skills Match | 40 | Proportion of required skills present in resume |
| Experience Match | 25 | Years of experience and role relevance |
| Education Match | 15 | Degree level and field of study |
| Tools & MLOps | 20 | Cloud, DevOps, and MLOps tool coverage |
| Total | 100 | — |

### 4.4 Anti-Hallucination Rules

- All three prompts include explicit instructions: Do NOT assume skills not present in the resume
- Temperature 0.0 used for extraction and matching — deterministic, no creative inference
- JSON-only output format enforced to prevent free-text hallucination
- safe_parse_json() helper strips any accidental markdown fencing from LLM output

## 5. Results & Analysis

### 5.1 Scoring Summary

| Candidate | Skills (/40) | Experience (/25) | Education (/15) | Tools (/20) | Total (/100) | Grade | Recommendation |
|---|---|---|---|---|---|---|---|
| Priya Sharma (Strong) | 37 | 23 | 15 | 18 | 93 | A | Strongly Recommend |
| Rahul Verma (Average) | 18 | 12 | 10 | 6 | 46–52 | C | Consider |
| Anil Kumar (Weak) | 2 | 2 | 5 | 1 | 10–12 | D | Reject |

### 5.2 Candidate Explanations

#### Priya Sharma — Strong Candidate

Priya's profile is an exceptional match for the Data Scientist role. She demonstrates 4 years of directly relevant experience at Flipkart AI Labs, covering the full skill spectrum: Python, scikit-learn, XGBoost, LightGBM (ML), TensorFlow and PyTorch (DL), AWS SageMaker and GCP (cloud), and MLflow, Docker, Kubernetes (MLOps). Her M.Tech from IIT Bombay provides a strong educational foundation. The real-time recommendation engine project serving 10M+ users and the NLP fraud detection deployment are concrete evidence of production-level capability. Score: 93/100, Grade A — Strongly Recommend.

#### Rahul Verma — Average Candidate

Rahul shows foundational data science skills but has significant gaps relative to the job description. He covers Python, SQL, pandas, scikit-learn, and basic AWS, which satisfies roughly half the requirements. Critical missing areas include deep learning (no TensorFlow/PyTorch), advanced ML libraries (XGBoost/LightGBM not listed), MLOps tooling (no MLflow, Docker, Kubernetes), and statistical testing. His 2.5 years of experience as a Junior Data Analyst falls short of the 3–5 year requirement. Score: ~52/100, Grade C — Consider (with upskilling path).

### Anil Kumar — Weak Candidate

Anil's background is in web development (HTML, CSS, JavaScript) with only a beginner-level Python bootcamp and no data science experience. His B.Com degree does not align with the technical education requirement. The resume contains no ML, statistics, cloud, or MLOps skills. The pipeline correctly assigns a very low score, demonstrating that the anti-hallucination constraints are working — no skills are assumed based on the role he applied for. Score: ~12/100, Grade D — Reject.

## 6. LangSmith Tracing

### 6.1 Tracing Configuration

LangSmith tracing was enabled via environment variables as required by the assignment:

```
os.environ['LANGCHAIN_TRACING_V2'] = 'true'
os.environ['LANGCHAIN_PROJECT']    = 'AI-Resume-Screening'
os.environ['LANGCHAIN_API_KEY']    = '<langsmith_api_key>'
```

With tracing enabled, every .invoke() call on a LangChain chain is automatically captured, including the full prompt text, LLM response, latency, and token usage. No additional instrumentation code is required.

### 6.2 Traced Runs

The LangSmith dashboard (Project: AI-Resume-Screening) shows all runs executed during this assignment. Each call to screen_resume() generates a RunnableSequence trace with the three pipeline steps visible as child spans:

| Run | Candidate | Latency (approx) | Tokens Used | Status |
|---|---|---|---|---|
| Run 1 | Priya Sharma (Strong) | 0.59s–0.71s | ~913 | ✅ Success |
| Run 2 | Rahul Verma (Average) | 0.27s–0.67s | ~569–866 | ✅ Success |
| Run 3 | Anil Kumar (Weak) | 0.27s–0.37s | ~379–492 | ✅ Success |
| Debug Run | Unknown Person (Edge Case) | 0.30s | ~257 | ✅ Success (empty output) |

| Run | Candidate | Latency (approx) | Tokens Used | Status |
|---|---|---|---|---|
| Bonus Tagged Run | Priya Sharma (Tagged) | 0.52s | ~768 | ✅ Success |

### 6.3 Debugging — Edge Case

An intentional edge case was submitted: a vague resume with no specific skills ("I know computers and internet"). This demonstrated two debugging capabilities:

- The extraction chain correctly returned years_experience: 0 and an empty skills list — no hallucinated skills.
- The LangSmith trace captured the exact prompt sent and the minimal response returned, making it easy to verify the anti-hallucination behavior.
- The run is visible in LangSmith as the 'Unknown Person' trace with low token count (~257) and 0.30s latency.

### 6.4 LangSmith Screenshot

The LangSmith screenshot (LangSmith_Tracing.png) confirms the AI-Resume-Screening project with multiple RunnableSequence traces, all successful (green checkmarks), with latencies ranging from 0.27s to 0.71s and token counts from 257 to 913. All pipeline steps are visible per run.

## 7. Bonus Features Implemented

### 7.1 Few-Shot Prompting

A FewShotPromptTemplate was implemented for a quick-score chain, providing 3 labeled examples (strong/average/weak) to guide the model toward consistent score formats. This reduces format variance and improves score calibration across different resumes.

```
Examples provided:
  Profile: 5 years Python, ML expert, AWS certified → 88/100 — Grade A
  Profile: 2 years Python, basic sklearn, no cloud   → 52/100 — Grade C
  Profile: No ML skills, 6 months web dev, B.Com     → 12/100 — Grade D
```

### 7.2 Structured JSON Output

All three pipeline steps produce strictly structured JSON output, enforced through prompt constraints and parsed via safe_parse_json(). This enables programmatic downstream processing (e.g., inserting results into a database, rendering a dashboard, or sending automated email reports).

### 7.3 LangSmith Tags & Metadata

The bonus screen_resume_with_tags() function passes RunnableConfig with tags and metadata to each chain invocation. Tags such as resume-screening, strong-candidate, and data-scientist-role appear in the LangSmith dashboard, enabling efficient filtering and comparison across runs.

```
config = RunnableConfig(
    tags=["resume-screening", "strong-candidate", "data-scientist-role"],
    metadata={"candidate_type": "STRONG"}
)
```

## 8. LangChain LCEL Implementation

### 8.1 Chain Construction

All three chains follow the LCEL pipe operator pattern (prompt | llm), as required:

```
extraction_chain = extraction_prompt | get_llm(temperature=0.0)
matching_chain   = matching_prompt   | get_llm(temperature=0.0)
scoring_chain    = scoring_prompt    | get_llm(temperature=0.1)
```

### 8.2 Invocation Pattern

All chains are invoked using the .invoke() method with explicit keyword-argument dictionaries:

```
# Step 1
extraction_result = extraction_chain.invoke({"resume": resume})

# Step 2
matching_result = matching_chain.invoke({
    "extracted_profile": extracted_profile,
    "job_description": job_description
})

# Step 3
scoring_result = scoring_chain.invoke({
    "extracted_profile": extracted_profile,
    "match_analysis": match_analysis,
    "job_description": job_description
})
```

Every .invoke() call is automatically traced in LangSmith as a child span within the parent RunnableSequence run — no manual trace wrapping is required.

## 9. Assignment Requirements Checklist

| Requirement | Status | Notes |
|---|---|---|
| 3 Resumes (Strong, Average, Weak) | ✅ Complete | Priya Sharma, Rahul Verma, Anil Kumar |
| 1 Job Description | ✅ Complete | Data Scientist at TechCorp Analytics |
| Step 1: Skill Extraction | ✅ Complete | extraction_chain — JSON output |
| Step 2: Matching Logic | ✅ Complete | matching_chain — gap analysis JSON |
| Step 3: Scoring (0–100) | ✅ Complete | scoring_chain — rubric-based score |
| Step 4: Explanation | ✅ Complete | Included in scoring JSON output |
| PromptTemplate used | ✅ Complete | All 3 prompts use PromptTemplate |
| LCEL .invoke() method | ✅ Complete | All chains use prompt \| llm pattern |
| LANGCHAIN_TRACING_V2=true | ✅ Complete | Set in environment variables |
| 3+ LangSmith runs visible | ✅ Complete | Confirmed in screenshot |
| All pipeline steps traced | ✅ Complete | All spans visible per run |
| Debug: Incorrect/edge output | ✅ Complete | Unknown Person edge case shown |
| No hardcoded outputs | ✅ Complete | All outputs generated by LLM |
| No hallucinated assumptions | ✅ Complete | Anti-hallucination prompts used |
| Bonus: Few-shot prompting | ✅ Complete | FewShotPromptTemplate implemented |
| Bonus: Structured JSON output | ✅ Complete | All chains output valid JSON |
| Bonus: LangSmith tags | ✅ Complete | RunnableConfig with tags used |

## 10. Key Learnings

- **The pipe operator (prompt | llm) creates clean, readable, composable chains. Each step's output becomes the next step's input automatically.**LangChain LCEL:
- **By simply setting LANGCHAIN_TRACING_V2=true, every .invoke() call is logged with full input/output/latency/token data. This is invaluable for debugging LLM pipelines in production.**LangSmith Tracing:
- **Anti-hallucination instructions (Do NOT assume skills not present in the resume) are critical for evaluation systems where factual accuracy matters. Temperature 0.0 for extraction prevents creative inference.**Prompt Engineering:
- **Requiring JSON-only responses from the LLM and using safe_parse_json() to strip markdown fencing makes pipelines robust against LLM formatting inconsistencies.**Structured Outputs:

- **Including a scoring rubric in the prompt (40 pts for skills, 25 pts for experience, etc.) forces the LLM to produce transparent, auditable reasoning rather than opaque scores.**Explainability:
- **Groq's LLaMA-3.3-70B provides fast, free inference suitable for assignment and prototyping work, with latencies of 0.27s–0.71s per chain call.**Groq vs OpenAI:

---